So far, we've looked at dynamic mapping and seen that although this feature is attractive, it does have limitations, as discussed in the next section.

4.2.2 Limitations of dynamic mapping

There are some limitations to letting Elasticsearch derive the document schema. Elasticsearch can misinterpret document field values and derive an incorrect mapping that voids the field's eligibility for appropriate search, sort, and aggregation capabilities. Let's find out how Elasticsearch determines data types incorrectly and thus aids in formulating inaccurate and erroneous mapping rules.

DERIVING AN INCORRECT MAPPING

Suppose we intend to provide a field with numeric data but that the data will be wrapped as a string (for example, `"3.14"`—note the quotes). Unfortunately, Elasticsearch treats such data incorrectly: in this example, it expects the data type to be `text`, not `float` or `double`.

Let's modify the `student` document we looked at in chapter 3 and add a field: the student's age. It sounds like a number, but we're indexing it as a `text` field by wrapping the value in quotes.

Listing 4.2 Adding an `age` field with text values to the `student` document

```
PUT students_temp/_doc/1
{
  "name":"John",
  "age":"12"
}

PUT students_temp/_doc/2
{
  "name":"William",
  "age":"14"
}
```

The age variable's value is set as a string (in quotes).

Second document

First document

Notice that the `age` field's value is enclosed in quotes, so Elasticsearch treats it as a text field (although it includes numbers). After these two documents are indexed, we can write a search query to sort students by ascending or descending age (see figure 4.4). The response indicates that the operation is not allowed; let's see why.

Because the `age` field of a `student` document is indexed as a `text` field (by default, any string field is mapped to a `text` field during the dynamic mapping process), Elasticsearch cannot use that field in sorting operations. Sorting functionality is not available on `text` fields by default.